Today 9:24 AM

PDF **Lab_4_VibeCoding...**
PDF

Scenario 21 — Checkers

A modular terminal checkers game with a board, movement rules, captures, and promotion support.

Provided files

- `main.py` — entry point.
- `game.py` — turn flow and interaction.
- `board.py` — board representation and movement.
- `rules.py` — legal-move and promotion rules.
- `requirements.txt` — dependency declaration.

Setup

python main.py

Before changing the code

Play enough turns to understand ordinary moves and captures. Set up a capture position and inspect the board before and after the move.

Task 1 — Capture correctness

When a legal capture is made, remove the opponent piece that was jumped. Preserve the moving piece and its destination.
Done when: captured pieces disappear and cannot be captured again.

Task 2 — Win and no-move detection

Detect when one side has no pieces or no legal moves. End the game cleanly in either case.

Task 3 — Complete checkers rules

Add forced-capture behaviour, multi-capture turns, and king movement in both directions. Promotion must occur at the correct

back row.

Task 4 — Move-level feedback

Add concise move/capture/promotion feedback. One accepted move should produce one player-facing action result, even if several internal legal-move checks occur.

Required testing

Test ordinary moves, captures, multiple captures, forced captures, promotion, king movement, no-move positions, invalid coordinates, and quitting.

LLM usage

You may use an LLM during the lab. The goal is to use it as a coding assistant while retaining responsibility for understanding and testing the result.

- Inspect the existing code before asking for changes.
- Ask for explanations when you do not understand a proposed change.
- Test generated code against the stated behaviour and edge cases.
- Keep your complete LLM chat history for submission.
- Do not replace the whole project with an unrelated implementation.
- Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.

Submission checklist

- Task 1 completed and the original defect was reproduced and fixed.
- Tasks 2–4 completed and tested.
- Boundary and invalid-input cases tested.
- No unnecessary external dependencies added.
- No persistent storage added.
- Code remains understandable and modular.
- Complete LLM chat-history link included.

Folder structure

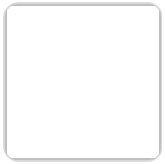
```
scenario-09-checkers/
├── README.md
├── requirements.txt
├── main.py
├── game.py
├── board.py
└── rules.py
```

Submission Checklist

Submission is only the following three things:

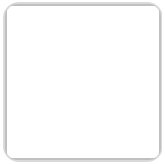
- A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- The Chat/LLM used page link, with the complete chat history

guide me step by step



how to see recorded video?

next



0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B

3 . . . . . .
4 . R . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
B> 2 1 3 0

0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . B . B . B
3 B . . . . . .
4 . R . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 1 2 2 1
That is not your piece.

0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . B . B . B
3 B . . . . . .
4 . R . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 2 3 3 2
That is not your piece.

0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . B . B . B
3 B . . . . . .
4 . R . . . . .

```
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R>
```

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . B . B . B
3 B . . . . . .
4 . R . R . . .
5 . . . . R . R .
6 . R . R . R . R
7 R . R . R . R .
B>
```

```
B> 2 3 3 4
```

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . . . B . B
3 B . . . B . . .
4 . R . R . . . .
5 . . . . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 4 3 2 5
Invalid move.
```

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . . . B . B
3 B . . . B . . .
4 . R . R . . . .
5 . . . . R . R .
```

```
6  . R . R . R . R
7  R . R . R . R .
R>
```

```
  def simple_move(board, player, start, end):
      sr, sc = start
      er, ec = end
      direction = -1 if player == "R" else 1
      return (
          board[er][ec] == "." and
          abs(er - sr) == 1 and abs(ec - sc) == 1 and
          er - sr == direction
      )


SIZE = 8


def initial_board():
    board = [["."] * SIZE for _ in range(SIZE)]
    for r in range(3):
        for c in range(SIZE):
            if (r + c) % 2 == 1:
                board[r][c] = "B"
    for r in range(5, 8):
        for c in range(SIZE):
            if (r + c) % 2 == 1:
                board[r][c] = "R"
    return board


def move_piece(board, start, end):
    board[end[0]][end[1]] = board[start[0]][start[1]]
    board[start[0]][start[1]] = "."
```

```
0 1 2 3 4 5 6 7
0  . B . B . B . B
```

```
1  B . B . B . B .
2  . . . B . B . B
3  B . . . . . .
4  . R . R . . . .
5  . . . . R . R .
6  . R . R . R . R
7  R . R . R . R .
B> 2 3 3 4

   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . . . . . B . B
3  B . . . B . . .
4  . R . R . . . .
5  . . . . R . R .
6  . R . R . R . R
7  R . R . R . R .
R> 4 3 2 5
Invalid move.

   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . . . . . B . B
3  B . . . B . . .
4  . R . R . . . .
5  . . . . R . R .
6  . R . R . R . R
7  R . R . R . R .
R> Traceback (most recent call last):
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\main.py", line 4,
in <module>
    Checkers().run()
    ~~~~~~~~~~~~~~~^^
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\game.py", line
19, in run
```

```
    raw = input(f"{self.player}> ").strip().lower().split()
          ~~~~~^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python main.py
Traceback (most recent call last):
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\main.py", line 1,
in <module>
    from game import Checkers
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\game.py", line 1,
in <module>
    from board import initial_board, move_piece, SIZE
ImportError: cannot import name 'initial_board' from 'board'
(C:\Users\Admin\Desktop\SE_ass\21_checkers\board.py)
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

```
R . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R> 4 3 2 5
That is not your piece.

   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R>
```

```
from board import initial_board, move_piece, SIZE
from rules import simple_move, capture_move, promote


class Checkers:
```

```
def __init__(self):
    self.board = initial_board()
    self.player = "R"

def print_board(self):
    print("\n   " + " ".join(str(c) for c in range(SIZE)))
    for r, row in enumerate(self.board):
        print(f"{r}  " + " ".join(row))

def run(self):
    print("Checkers — move: sr sc er ec")
    while True:
        self.print_board()
        raw = input(f"{self.player}> ").strip().lower().split()
        if raw == ["q"]:
            return
        if len(raw) != 4:
            print("Enter four coordinates.")
            continue
        try:
            sr, sc, er, ec = map(int, raw)
        except ValueError:
            print("Coordinates must be numbers.")
            continue
        if not all(0 <= x < SIZE for x in (sr, sc, er, ec)):
            print("Outside board.")
            continue
        if self.board[sr][sc] not in (self.player, self.player + "K"):
            print("That is not your piece.")
            continue

        start, end = (sr, sc), (er, ec)
        if capture_move(self.board, self.player, start, end):
            move_piece(self.board, start, end)
        elif simple_move(self.board, self.player, start, end):
            move_piece(self.board, start, end)
        else:
```

```
    print("Invalid move.")
    continue

promote(self.board)
self.player = "B" if self.player == "R" else "R"


        SIZE = 8


        def simple_move(board, player, start, end):
            sr, sc = start
            er, ec = end
            direction = -1 if player == "R" else 1
            return (
                board[er][ec] == "." and
                abs(er - sr) == 1 and abs(ec - sc) == 1 and
                er - sr == direction
            )


        def capture_move(board, player, start, end):
            sr, sc = start
            er, ec = end
            direction = -1 if player == "R" else 1
            mr, mc = (sr + er) // 2, (sc + ec) // 2
            return (
                board[er][ec] == "." and
                abs(er - sr) == 2 and abs(ec - sc) == 2 and
                er - sr == 2 * direction and
                board[mr][mc] not in (".", player)
            )


        def promote(board):
            for c in range(SIZE):
                if board[0][c] == "R":
                    board[0][c] = "RK"
```

```
            if board[SIZE - 1][c] == "B":
                board[SIZE - 1][c] = "BK"


    from rules import capture_move

    assert capture_move(board, "R", (4, 3), (2, 5)), \
        "A valid capture was rejected"
```

give this combined code

```
Board after capture:
. . . . . . . .
. . . . . . . .
. . . . . R . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .

PASS: Capture validation works.
PASS: Captured piece was removed.
PASS: Red reached the destination.
PASS: Starting square was cleared.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

next

give me complete what cheges needed

```
insertions(+), 15 deletions(-)
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python main.py
Traceback (most recent call last):
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\main.py", line 1,
in <module>
```

```
    from game import Checkers
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\game.py", line 2,
in <module>
    from board import initial_board, move_piece, SIZE
ImportError: cannot import name 'initial_board' from 'board'
(C:\Users\Admin\Desktop\SE_ass\21_checkers\board.py)
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

```
S C:\Users\Admin\Desktop\SE_ass\21_checkers> python main.py
Checkers — move: sr sc er ec (or q to quit)

  0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R>
```

```
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python test_win.py
PASS: remaining pieces detected.
PASS: no-pieces condition detected.
PASS: no-legal-moves condition detected.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

Today 6:14 PM

```
uch file or directory
PS C:\Users\Admin\Desktop\SE_ass> cd 21_checkers
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python test_win.py
PASS: remaining pieces detected.
PASS: no-pieces condition detected.
```

PASS: no-legal-moves condition detected.

PS C:\Users\Admin\Desktop\SE_ass\21_checkers>

python test_win.py

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . . . . . . . .
5 R . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R>
```

Checkers — move: sr sc er ec (or q to quit)

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . . . . . . . .
5 R . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 5 0 4 1
```

R moves to (4, 1).

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . R . . . . . .
```

```
5  . . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
B> 2 1 3 0
B moves to (3, 0).

   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . . . B . B . B
3  B . . . . . . .
4  . R . . . . . .
5  . . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R>
```

yboardInterrupt
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python test_rules.py
PASS: Capture availability detected.
PASS: Legal capture accepted.
PASS: Ordinary move validated.
PASS: Captured piece removed.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>

C:\Users\Admin\Desktop\SE_ass\21_checkers> python test_forced_capture.py
PASS: Capture is available.
PASS: Legal capture is recognized.
PASS: Ordinary move is geometrically valid.
NOTE: game.py must reject that ordinary move.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>

Ordinary move is geometrically valid.
NOTE: game.py must reject that ordinary move.

```
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_task3.py
PASS: Regular capture works.
PASS: Follow-up capture detected.
PASS: King can move backward.
PASS: Promotion works.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . . . . . . . .
5 R . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R>
```

```
from board import initial_board, move_piece, SIZE
from rules import (
    simple_move,
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture,
)


class Checkers:
    def __init__(self):
        self.board = initial_board()
        self.player = "R"
        self.continue_capture = False

    def print_board(self):
```

```
        print("\n   " + " ".join(str(c) for c in range(SIZE)))
        for r, row in enumerate(self.board):
            print(f"{r}  " + " ".join(row))

    def run(self):
        print("Checkers — move: sr sc er ec (or q to quit)")

        while True:
            self.print_board()
            raw = input(f"{self.player}> ").strip().lower().split()

            if raw == ["q"]:
                print("Game ended.")
                return

            if len(raw) != 4:
                print("Enter four coordinates.")
                continue

            try:
                sr, sc, er, ec = map(int, raw)
            except ValueError:
                print("Coordinates must be numbers.")
                continue

            if not all(0 <= x < SIZE for x in (sr, sc, er, ec)):
                print("Outside board.")
                continue

            start, end = (sr, sc), (er, ec)
            piece = self.board[sr][sc]

            if piece not in (self.player, self.player + "K"):
                print("That is not your piece.")
                continue

            if self.continue_capture and start != self.last_capture_position:
```

```
            print("Continue capturing with the same piece.")
            continue

        must_capture = has_capture(self.board, self.player)

        if must_capture and not capture_move(
            self.board, self.player, start, end
        ):
            print("A capture is available. You must capture.")
            continue

        is_capture = capture_move(
            self.board, self.player, start, end
        )

        if is_capture:
            move_piece(self.board, start, end)
            print(f"{self.player} captures a piece.")

            # Promote before checking for another jump.
            promote(self.board)

            self.last_capture_position = end

            if has_capture(self.board, self.player, end):
                self.continue_capture = True
                print("Another capture is available. Continue your turn.")
                continue

            self.continue_capture = False

        elif simple_move(self.board, self.player, start, end):
            if self.continue_capture:
                print("You must finish the capture sequence.")
                continue

            move_piece(self.board, start, end)
```

```
            print(f"{self.player} moves to ({er}, {ec}).")
            promote(self.board)

        else:
            print("Invalid move.")
            continue

        # The turn ends only after the move sequence is complete.
        self.player = "B" if self.player == "R" else "R"

        if not has_pieces(self.board, self.player):
            winner = "R" if self.player == "B" else "B"
            self.print_board()
            print(f"{winner} wins! The opponent has no pieces left.")
            return

        if not has_legal_move(self.board, self.player):
            winner = "R" if self.player == "B" else "B"
            self.print_board()
            print(f"{winner} wins! The opponent has no legal moves.")
            return
```

please finish it assesment fast???

```
File "C:\Users\Admin\Desktop\SE_ass\21_checkers\game.py", line 30,
in run
    raw = input(f"{self.player}> ").strip().lower().split()
          ~~~~~^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git status
>> git log --oneline -5
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
```

```
  (use "git restore <file>..." to discard changes in working directory)
        modified:   __pycache__/board.cpython-314.pyc
        modified:   __pycache__/game.cpython-314.pyc
        modified:   __pycache__/rules.cpython-314.pyc
        modified:   board.py
        modified:   game.py
        modified:   rules.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test_capture.py
        test_forced_capture.py
        test_game_rules.py
        test_rules.py
        test_task3.py
        test_win.py

no changes added to commit (use "git add" and/or "git commit -a")
8d0459c (HEAD -> main) Add win and no-move detection
82d445a Fix captured piece removal
030aa47 (origin/main, origin/HEAD) Updated scenario title
47a7518 Initial commit
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

```
__pycache__/
*.py[cod]
```

how?

```
.gitignore
        test_capture.py
        test_forced_capture.py
        test_game_rules.py
        test_rules.py
        test_task3.py
        test_win.py
```

```
 no changes added to commit (use "git add" and/or "git commit -a")
 PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

then

```
test_forced_capture.py
        test_game_rules.py
        test_rules.py
        test_task3.py
        test_win.py

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git add board.py
rules.py
>> git commit -m "Add capture chains and king promotion rules"
[main 362c500] Add capture chains and king promotion rules
 2 files changed, 71 insertions(+), 38 deletions(-)
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git add game.py
.gitignore
>> git commit -m "Add move feedback and ignore Python cache
files"
[main cccf9a3] Add move feedback and ignore Python cache files
 2 files changed, 67 insertions(+), 18 deletions(-)
 create mode 100644 .gitignore
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git add
test_capture.py test_forced_capture.py test_game_rules.py
test_rules.py test_task3.py test_win.py
>> git commit -m "Add checkers rule tests"
[main 5931933] Add checkers rule tests
 6 files changed, 183 insertions(+)
 create mode 100644 test_capture.py
 create mode 100644 test_forced_capture.py
 create mode 100644 test_game_rules.py
 create mode 100644 test_rules.py
 create mode 100644 test_task3.py
 create mode 100644 test_win.py
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git push origin
```

```
main
Enumerating objects: 27, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 8 threads
Compressing objects: 100% (22/22), done.
Writing objects: 100% (23/23), 6.77 KiB | 866.00 KiB/s, done.
Total 23 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), completed with 1 local object.
To https://github.com/pandu2006-28/21_checkers
   030aa47..5931933  main -> main
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_capture.py
>> python test_forced_capture.py
>> python test_game_rules.py
>> python test_rules.py
>> python test_task3.py
>> python test_win.py
Board before capture:
. . . . . .
. . . . . .
. . . . . .
. . . . B . . .
. . . R . . . .
. . . . . .
. . . . . .
. . . . . .

Board after capture:
. . . . . .
. . . . . .
. . . . . R . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
```

```
PASS: Capture validation works.
PASS: Captured piece was removed.
PASS: Red reached the destination.
PASS: Starting square was cleared.
PASS: Capture is available.
PASS: Legal capture is recognized.
PASS: Ordinary move is geometrically valid.
NOTE: game.py must reject that ordinary move.
PASS: Forced-capture test position created.
PASS: Ordinary move is available geometrically.
PASS: Legal capture is available.
NOTE: Interactive enforcement must be verified separately.
PASS: Capture availability detected.
PASS: Legal capture accepted.
PASS: Ordinary move validated.
PASS: Captured piece removed.
PASS: Regular capture works.
PASS: Follow-up capture detected.
PASS: King can move backward.
PASS: Promotion works.
PASS: remaining pieces detected.
PASS: no-pieces condition detected.
PASS: no-legal-moves condition detected.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python main.py
Checkers — move: sr sc er ec (or q to quit)

  0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R> Traceback (most recent call last):
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\main.py", line 4,
```

```
in <module>
    Checkers().run()
    ~~~~~~~~~~~~~~~^^
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\game.py", line
30, in run
    raw = input(f"{self.player}> ").strip().lower().split()
          ~~~~~^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git push origin
main
Everything up-to-date
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>


7  R . R . R . R .
R> Traceback (most recent call last):
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\main.py", line 4,
in <module>
    Checkers().run()
    ~~~~~~~~~~~~~~~^^
  File "C:\Users\Admin\Desktop\SE_ass\21_checkers\game.py", line
30, in run
    raw = input(f"{self.player}> ").strip().lower().split()
          ~~~~~^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git push origin
main
Everything up-to-date
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python main.py
Checkers — move: sr sc er ec (or q to quit)

   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . R . R . R .
```

```
6  . R . R . R . R
7  R . R . R . R .
R> q
Game ended.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

its working , next?

```
Checkers — move: sr sc er ec (or q to quit)

   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R> q
Game ended.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python main.py
Checkers — move: sr sc er ec (or q to quit)

   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R> 5 0 4 1
R moves to (4, 1).

   0 1 2 3 4 5 6 7
```

```
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . R . . . . . .
5  . . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
B> q
Game ended.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_forced_capture.py
PASS: Capture is available.
PASS: Legal capture is recognized.
PASS: Ordinary move is geometrically valid.
NOTE: game.py must reject that ordinary move.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_task3.py
>> python test_game_rules.py
PASS: Regular capture works.
PASS: Follow-up capture detected.
PASS: King can move backward.
PASS: Promotion works.
PASS: Forced-capture test position created.
PASS: Ordinary move is available geometrically.
PASS: Legal capture is available.
NOTE: Interactive enforcement must be verified separately.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>


. . . R . . . .
. . . . . .
. . . . . .
. . . . . .

Board after capture:
. . . . . .
. . . . . .
```

. . . . . R . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

PASS: Capture validation works.
PASS: Captured piece was removed.
PASS: Red reached the destination.
PASS: Starting square was cleared.
PASS: Capture is available.
PASS: Legal capture is recognized.
PASS: Ordinary move is geometrically valid.
NOTE: game.py must reject that ordinary move.
PASS: Forced-capture test position created.
PASS: Ordinary move is available geometrically.
PASS: Legal capture is available.
NOTE: Interactive enforcement must be verified separately.
PASS: Capture availability detected.
PASS: Legal capture accepted.
PASS: Ordinary move validated.
PASS: Captured piece removed.
PASS: Regular capture works.
PASS: Follow-up capture detected.
PASS: King can move backward.
PASS: Promotion works.
PASS: remaining pieces detected.
PASS: no-pieces condition detected.
PASS: no-legal-moves condition detected.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git add game.py test_game_rules.py
>> git commit -m "Verify interactive forced capture rules"
>> git push origin main
On branch main
Your branch is up to date with 'origin/main'.

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   __pycache__/board.cpython-314.pyc
        modified:   __pycache__/game.cpython-314.pyc
        modified:   __pycache__/rules.cpython-314.pyc

no changes added to commit (use "git add" and/or "git commit -a")
Everything up-to-date


    from game import Checkers
    from rules import has_capture, simple_move, capture_move

    game = Checkers()
    game.board = [["."] * 8 for _ in range(8)]

    # Red has a capture available.
    game.board[4][1] = "R"
    game.board[3][2] = "B"
    game.board[5][4] = "R"

    assert has_capture(game.board, "R")

    # This ordinary move is geometrically valid.
    assert simple_move(game.board, "R", (5, 4), (4, 5))

    # This capture is legal.
    assert capture_move(game.board, "R", (4, 1), (2, 3))

    print("PASS: Forced-capture test position created.")
    print("PASS: Ordinary move is available geometrically.")
    print("PASS: Legal capture is available.")
    print("NOTE: Interactive enforcement must be verified separately.")


. . . . . .
. . . . . .
. . . . . .
```

. . . . B . . .
. . . R . . . .
. . . . . .
. . . . . .
. . . . . .

Board after capture:

. . . . . .
. . . . . .
. . . . . R . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .

PASS: Capture validation works.
PASS: Captured piece was removed.
PASS: Red reached the destination.
PASS: Starting square was cleared.
PASS: Capture is available.
PASS: Legal capture is recognized.
PASS: Ordinary move is geometrically valid.
NOTE: game.py must reject that ordinary move.
PASS: Forced-capture test position created.
PASS: Ordinary move is available geometrically.
PASS: Legal capture is available.
NOTE: Interactive enforcement must be verified separately.
PASS: Capture availability detected.
PASS: Legal capture accepted.
PASS: Ordinary move validated.
PASS: Captured piece removed.
PASS: Regular capture works.
PASS: Follow-up capture detected.
PASS: King can move backward.
PASS: Promotion works.
PASS: remaining pieces detected.

```
PASS: no-pieces condition detected.
PASS: no-legal-moves condition detected.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git add game.py
test_game_rules.py
>> git commit -m "Verify interactive forced capture rules"
>> git push origin main
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   __pycache__/board.cpython-314.pyc
        modified:   __pycache__/game.cpython-314.pyc
        modified:   __pycache__/rules.cpython-314.pyc

no changes added to commit (use "git add" and/or "git commit -a")
Everything up-to-date
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   __pycache__/board.cpython-314.pyc
        modified:   __pycache__/game.cpython-314.pyc
        modified:   __pycache__/rules.cpython-314.pyc

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_game_rules.py
PASS: Interactive forced-capture enforcement works.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> git add
test_game_rules.py
>> git commit -m "Test interactive forced capture enforcement"
>> git push origin main
```

```
[main 94fda88] Test interactive forced capture enforcement
 1 file changed, 23 insertions(+), 12 deletions(-)
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 743 bytes | 743.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/pandu2006-28/21_checkers
   5931933..94fda88  main -> main
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>

PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_task3.py
PASS: Regular capture works.
PASS: Follow-up capture detected.
PASS: King can move backward.
PASS: Promotion works.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>

from board import initial_board, move_piece, SIZE
from rules import (
    simple_move,
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture,
)


class Checkers:
    def __init__(self):
        self.board = initial_board()
        self.player = "R"
        self.continue_capture = False
```

```
        self.last_capture_position = None

    def print_board(self):
        print("\n   " + " ".join(str(c) for c in range(SIZE)))
        for r, row in enumerate(self.board):
            print(f"{r}  " + " ".join(row))

    def run(self):
        print("Checkers — move: sr sc er ec (or q to quit)")

        while True:
            self.print_board()
            raw = input(f"{self.player}> ").strip().lower().split()

            if raw == ["q"]:
                print("Game ended.")
                return

            if len(raw) != 4:
                print("Enter four coordinates.")
                continue

            try:
                sr, sc, er, ec = map(int, raw)
            except ValueError:
                print("Coordinates must be numbers.")
                continue

            if not all(0 <= x < SIZE for x in (sr, sc, er, ec)):
                print("Outside board.")
                continue

            start, end = (sr, sc), (er, ec)
            piece = self.board[sr][sc]

            if piece not in (self.player, self.player + "K"):
                print("That is not your piece.")
```

```
            continue

        if self.continue_capture and start != self.last_capture_position:
            print("Continue capturing with the same piece.")
            continue

        is_capture = capture_move(
            self.board, self.player, start, end
        )

        # Captures are compulsory.
        if has_capture(self.board, self.player) and not is_capture:
            print("A capture is available. You must capture.")
            continue

        if not is_capture and not simple_move(
            self.board, self.player, start, end
        ):
            print("Invalid move.")
            continue

        # Record whether this move causes promotion.
        moving_piece = self.board[sr][sc]
        move_piece(self.board, start, end)
        promote(self.board)
        promoted = (
            moving_piece in ("R", "B")
            and self.board[er][ec] in ("RK", "BK")
        )

        # Check for another capture before ending the turn.
        if is_capture and has_capture(
            self.board, self.player, end
        ):
            self.continue_capture = True
            self.last_capture_position = end
```

```
            if promoted:
                print(
                    f"{self.player} captures and promotes to "
                    f"{self.board[er][ec]}."
                )
            else:
                print(f"{self.player} captures a piece.")

            print("Another capture is available; continue your turn.")
            continue

        # One concise result for the accepted move.
        if is_capture and promoted:
            print(
                f"{self.player} captures and promotes to "
                f"{self.board[er][ec]}."
            )
        elif is_capture:
            print(f"{self.player} captures a piece.")
        elif promoted:
            print(
                f"{self.player} moves to ({er}, {ec}) "
                f"and promotes to {self.board[er][ec]}."
            )
        else:
            print(f"{self.player} moves to ({er}, {ec}).")

        self.continue_capture = False
        self.last_capture_position = None

        # Switch turns only after the move sequence finishes.
        self.player = "B" if self.player == "R" else "R"

        if not has_pieces(self.board, self.player):
            winner = "R" if self.player == "B" else "B"
            self.print_board()
            print(f"{winner} wins! The opponent has no pieces left.")
```

```
            return

        if not has_legal_move(self.board, self.player):
            winner = "R" if self.player == "B" else "B"
            self.print_board()
            print(f"{winner} wins! The opponent has no legal moves.")
            return
```

```
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 743 bytes | 743.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/pandu2006-28/21_checkers
   5931933..94fda88  main -> main
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_task3.py
PASS: Regular capture works.
PASS: Follow-up capture detected.
PASS: King can move backward.
PASS: Promotion works.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_multicapture_interactive.py
Traceback (most recent call last):
  File
"C:\Users\Admin\Desktop\SE_ass\21_checkers\test_multicapture_inte
ractive.py", line 30, in <module>
    assert "same piece" in output.lower(), "Wrong-piece continuation
was not rejected."
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: Wrong-piece continuation was not rejected.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

```
call last):
  File
"C:\Users\Admin\Desktop\SE_ass\21_checkers\test_multicapture_inte
ractive.py", line 36, in <module>
```

```
    assert "same piece" in output.lower(), "Wrong-piece continuation was not rejected."
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: Wrong-piece continuation was not rejected.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

```
7  . . . . . .
That is not your piece.

  0 1 2 3 4 5 6 7
0  . . . . . .
1  . . . . B . . .
2  . . . R . . . .
3  . . . . . .
4  . . . . . .
5  . . . . . .
6  . . . . . .
7  . . . . . .
R captures and promotes to RK.

  0 1 2 3 4 5 6 7
0  . . . . . RK . .
1  . . . . . .
2  . . . . . .
3  . . . . . .
4  . . . . . .
5  . . . . . .
6  . . . . . .
7  . . . . . .
R wins! The opponent has no pieces left.
Did the wrong-piece move get rejected?
False
PASS: Interactive multi-capture works.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers>
```

please complete it fast i dont have time

```
from unittest.mock import patch
from game import Checkers

game = Checkers()
game.board = [["."] * 8 for _ in range(8)]

# Set up a two-capture sequence for Red.
game.board[4][1] = "R"
game.board[3][2] = "B"
game.board[1][4] = "B"

# Attempt first capture, wrong-piece move, then second capture.
inputs = [
    "4 1 2 3",
    "5 0 4 1",
    "2 3 0 5",
    "q",
]

with patch("builtins.input", side_effect=inputs):
    with patch("builtins.print") as mock_print:
        game.run()

output = "\n".join(
    " ".join(str(arg) for arg in call.args)
    for call in mock_print.call_args_list
)

print("Captured output:")
print(output)

assert game.board[3][2] == ".", "First captured piece remains."
assert game.board[1][4] == ".", "Second captured piece remains."
assert game.board[0][5] == "RK", "Red did not promote after the
capture chain."
assert game.player == "B", "Turn did not switch after the capture
chain."
```

```
print("Did the wrong-piece move get rejected?")
print("Continue capturing with the same piece." in output)

print("PASS: Interactive multi-capture works.")
```

give complete code to paste

```
0  . . . . . RK . .
1  . . . . . . . .
2  . . . . . . . .
3  . . . . . . . .
4  . . . . . . . .
5  . . . . . . . .
6  . . . . . . . .
7  . . . . . . . .
R wins! The opponent has no pieces left.
Did the wrong-piece move get rejected?
False
PASS: Interactive multi-capture works.
PS C:\Users\Admin\Desktop\SE_ass\21_checkers> python
test_multicapture_interactive.py
PASS: First capture removes the captured piece.
PASS: Different piece is rejected during the capture chain.
PASS: Second capture completes successfully.
PASS: Promotion works after the capture chain.
PASS: Turn switches after the capture chain.
```

next

how to take after video

okay next